

---

# Lumin: A Hybrid Retrieval and Reasoning Framework for Academic Assistance

---

**Berke Kara\***

Department of Computer Science  
The University of Texas at Austin  
Austin, TX 78712  
bkara@cs.utexas.edu

## Abstract

Students today rely heavily on AI to study, yet the tools available to them share a common flaw: they produce answers without making their reasoning visible. A student who cannot trace how an answer was reached cannot verify it, cannot investigate further, and cannot develop the conceptual understanding that distinguishes learning from information retrieval. We introduce **Lumin**, an educational assistant designed to support student research and study. Lumin ingests multimodal course materials (lecture notes, slide decks, textbook PDFs, and videos) and organizes them into a hierarchical concept knowledge graph and vector database. When a student asks a question, Lumin routes the query through dense semantic retrieval, graph-based traversal, or a hybrid of both, verifies the relevance and grounding of its own answers using Self-RAG reflection tokens, and renders the retrieved subgraph as an interactive mind-map that highlights the exact reasoning path used.

## 1 Introduction

With the advancement of Large Language Models (LLMs), the tools students use to aid their learning have evolved significantly. Walk into any university library today and you are likely to find students with two windows open: their assignments, and an AI chatbot. A 2024 survey found that 86% of students globally use AI during their studies, and by 2025, 84% of U.S. high school students reported using generative AI for schoolwork [1, 2]. Of those using AI, the most common use case (reported by 69%) was information retrieval [1].

This shift is understandable. Large language models (LLMs) such as GPT-5, Claude, and Gemini can explain difficult concepts, synthesize ideas across topics, and answer follow-up questions with ease. But they come with big drawbacks as they hallucinate, produce claims unsupported by course content, and degrade in quality as students fill a conversation with notes and questions, a phenomenon known as “lost in the middle,” where information near the ends of a long context is attended to far more than information in the middle.

Retrieval-augmented generation (RAG) was developed precisely to address these limitations. Instead of relying only on a model’s pre-trained knowledge, a RAG system retrieves relevant passages from an external knowledge store and conditions its answers on that evidence. This allows factual grounding and makes it possible to build tools that are specific to a particular course, instructor, or set of materials.

---

\*Project completed under the mentorship of Professor Fares Fraij, Department of Computer Science, The University of Texas at Austin.

Yet RAG, in its standard form, falls short in an educational setting in three specific ways. *First*, retrieval happens once, before the generation, and cannot adapt to the chain reasoning. Questions that require connecting ideas across multiple lectures, “How does the chain rule relate to the gradient computation?” require iterative evidence gathering that a single retrieval step cannot support. *Second*, standard retrieval treats each passage in isolation, which means it struggles to answer questions whose answers emerge from the *relationship* between concepts, not the content of any single passage. *Third*, and most importantly for education, RAG systems give students no way to see *why* an answer was produced. A student who reads a generated paragraph and cannot tell whether it came from the textbook, the lecture slides, or the model’s imagination.

GraphRAG [5] partially addresses the second of these problems by organizing a corpus into entities, relations, and hierarchical community summaries, enabling reasoning over the structure of a knowledge base rather than just its content. But it still retrieves once and verifies nothing, and the graph it produces is optimized for general information extraction, not for the pedagogical distinctions such as prerequisites, definitions, examples, counterexamples that is specific to relationships students learn.

We introduce **Lumin**, a system that addresses all three of these gaps simultaneously. Lumin’s three core contributions are:

1. **A course-content knowledge graph** constructed from multimodal sources and organized around meaningful relationship types. Rather than flattening lecture videos, slide decks, and PDFs into a single text index, Lumin preserves each source’s structure and encodes concepts in a three-level hierarchy (Topic → Subtopic → Content) with edges created by LLM assistance to personalize for each content.
2. **An adaptive retrieval router** that selects between dense retrieval, graph-based retrieval, and a hybrid of both based on the query, and wraps each hop in Self-RAG reflection gates that verify relevance and grounding before any answer is returned.
3. **An interactive mind-map visualization** that renders the retrieved subgraph with the reasoning path highlighted, and that deep-links every cited claim back to the exact page or timestamp in the original source material. Figure 3

This paper presents a proof-of-concept system for academic assistance, detailing its architecture and design decisions. We demonstrate how combining knowledge graphs with a dense vector store and adaptive retrieval improves grounding and reduces hallucinations.

## 2 Background and Motivation

### 2.1 Retrieval-Augmented Generation

Retrieval-augmented generation, introduced by Lewis et al. [3], combines a neural language model with a retriever that extracts relevant passages from an external corpus using dense embeddings. By grounding generation in retrieved evidence, RAG substantially reduces hallucination and enables systems that remain current without retraining. Subsequent improvements such as dense passage retrievers like DPR [6], cross-encoder re-rankers [7], and instruction-tuned generators have brought RAG to a high level of capability for many knowledge-intensive tasks.

However, RAG fails with complex questions that require relationship heavy information where retrieval needs to connect disjointed information to answer a users query. Standard top-k similarity search returns the passages most textually similar to the query, which tend to cluster around a single concept, and cannot synthesize the conceptual structure that makes an answer educationally useful. When multiple hops required to answer users query, RAG fails to retrieve correct chunks.

Furthermore, Self-RAG [4] was introduced to increase grounding in RAG systems by training a model to emit special **reflection tokens** during generation. These tokens signal allow LLM to decide if it needs to retrieve, whether retrieved passages are relevant, and whether each generated sentence is supported by the evidence. Lumin implements this concept by using Self-RAG-style reflection gates to verify each hop during the retrieval process before the result is returned to the student.

## 2.2 Graph-Based Retrieval and Its Limits

GraphRAG [5] is a recent advancement that solved limitations of RAG systems by extracting entities, relations, and hierarchical summaries from source documents and storing them in a graph database.

For educational content, however, GraphRAG’s general-purpose relational schema is a substantive limitation. Knowing that concept A is “related to” concept B is less useful during retrieval when the system traverses the graph database based on the type of relationship. The distinctions between relationships enable more informed retrieval decisions and allow students to view the specific relationships they have. Lumin additionally uses the Self-RAG reflection concept, allowing the system to re-query when it determines that the retrieved subgraph is insufficient.

## 2.3 Visualization and Learning

A recurring finding in educational technology research is that students benefit most from systems that make reasoning legible. Work on concept-map-based tutoring has shown that interactive, query-specific concept visualizations improve both comprehension and metacognitive awareness [8]. Yet modern RAG-based course assistants almost universally surface their reasoning only as retrieved text snippets alongside a generated answer.

Lumin allows students to see how concepts connect by rendering the retrieved subgraph as a navigable mind map, showing the nodes and edges used to answer the current question. Every cited claim links back to the original source. Students can inspect, navigate, and explore the system’s reasoning, giving them more control during their study sessions.

# 3 System Design

## 3.1 Guiding Principles

Students come across information through many multimodal sources, such as a slide’s caption, a video’s transcript, and a textbook’s equations, each carrying information in different forms and at different levels of granularity. Lumin preserves modality through a unified Chunk abstraction all the way from ingestion to the user interface. With Lumin, the source for each chunk is also preserved, so the user is able to point back to the original content for deeper exploration.

Additionally, different knowledge stores, such as dense vector and graph databases, both offer advantages while having limitations. Some questions are best answered by finding the passage most semantically similar to the query, while others are best answered by traversing a relational graph. Many require both. Lumin exposes three retrieval paths—dense, graph, and hybrid—and a router that decides between them based on the user’s query.

Following the Self-RAG concept, Lumin verifies both the relevance of what it retrieves and the grounding of what it generates. This verification is surfaced to the students. A compact confidence badge on each answer tells the student when the system is uncertain, enabling them to treat the answer accordingly.

## 3.2 Architecture Overview

Lumin is implemented as a Python FastAPI backend and a TypeScript React frontend connected over a JSON API. The backend is organized around five modules: (1) **ingestion**, which processes source materials into modality-specific Chunk objects; (2) **db**, which persists chunks in a SQLite WAL store; (3) **graph**, which synthesizes a hierarchical concept graph from the chunk collection; (4) **retrieval**, which provides dense, graph, and hybrid access paths; and (5) **selfrag**, which orchestrates multi-hop reasoning with reflection gates. On the frontend, a Cytoscape.js-based graph panel sits beside a conversational pane; selecting any highlighted node opens a detail panel showing the node’s attributes and the raw evidence chunks behind it.

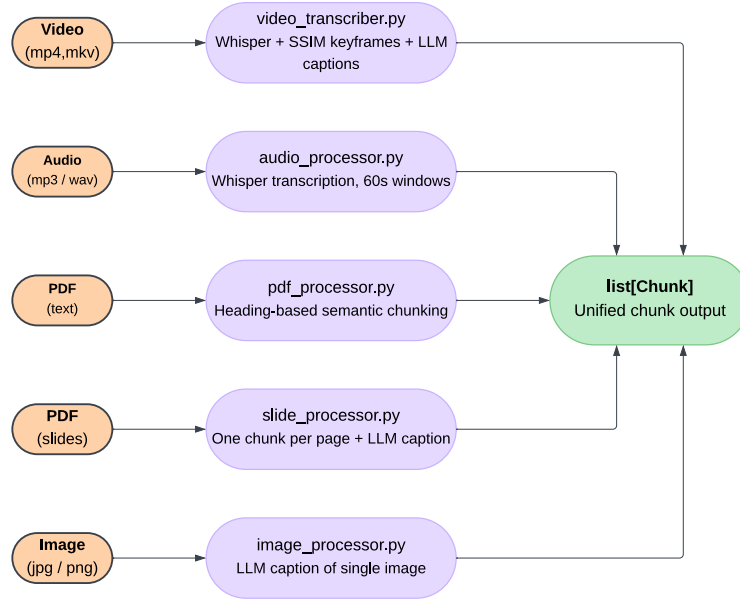


Figure 1: The Lumin ingestion pipeline. Each source is routed to a dedicated chunker that preserves structure (section boundaries for PDFs, page units for slides, transcript(keyframe alignment for video)) before chunks are persisted in a unified SQLite store. Embeddings flow into a FAISS index, while the chunk collection feeds the four-stage graph construction pipeline.

## 4 Multimodal Ingestion Pipeline

### 4.1 Modality-Specific Chunking

Lumin applies a dedicated chunker to each modality, producing `Chunk` objects that carry: the raw text, a modality tag (`pdf`, `slide`, `video`, `audio`, `image`), the source filename, a modality-specific address (page number or timestamp), and additional metadata such as captions or keyframe paths. Figure 1 summarizes the full pipeline from raw source files through modality-specific processing to the unified chunk store and graph builder.

**PDF documents** are processed with heading-aware semantic segmentation: a parser detects structural markers (bold lines, font-size increases, numbered section headers) and splits on section boundaries rather than token count. This preserves the logical structure of textbook chapters and course notes, ensuring that a retrieved chunk corresponds to a coherent conceptual unit rather than an arbitrary window of text.

**Slide decks** are converted to PDF for uniform handling, then split one chunk per page. Each page’s text is extracted; for pages that are predominantly visual, a captioning model generates a descriptive text chunk.

**Videos** are processed by aligning a `faster-whisper` transcript with keyframes detected using OpenCV scene-change detection. Each transcript segment is aligned to the nearest keyframe, so that the spoken content and the visual content of a lecture slide travel together in a single `Chunk`. The timestamp address allows the frontend to deep-link directly to the relevant moment in the video file.

**Audio recordings** are transcribed using `faster-whisper`.

**Images** are passed through the captioning model and stored as standalone text chunks carrying the image path as metadata.

### 4.2 Persistence

Chunks are stored in a SQLite database running in WAL (write-ahead logging) mode, which allows readers (query handlers) and writers (ingestion jobs) to operate concurrently without locking. Each

chunk is keyed by a chunk ID derived from a hash of the source filename, modality, and address, so that re-ingestion of an updated file produces consistent identifiers. This allows system to save space and computation by not allowing duplicate contents. Dense retrieval embeddings are persisted in a FAISS index backed by L2-normalized vectors, alongside a sidecar JSON file that maps FAISS integer indices to chunk IDs.

## 5 Hierarchical Concept Graph

### 5.1 Graph Structure and Schema

The knowledge graph is the conceptual backbone of Lumin. It is a three-level hierarchy—**Topic** → **Subtopic** → **Content**—implemented as a NetworkX `MultiDiGraph`. The use of a `MultiDiGraph` allows multiple parallel edges between the same pair of nodes, each carrying a different semantic label. A Content node about the chain rule can be connected to one about gradient descent via both a `prerequisite_of` edge and a `used_in` edge. Collapsing these into a single edge would erase the relational specificity that graph retrieval is intended to exploit.

The schema distinguishes two classes of relations. **Structural relations**—`HAS_SUBTOPIC`, `HAS_CONTENT`, and `EVIDENCE_OF`, define the taxonomic hierarchy and are validated against a `frozenset` to prevent schema violations. **Semantic relations**, all of type `RELATED_TO` with a free-form `snake_case` label (e.g., `prerequisite_of`, `defined_by`, `example_of`, `contrasts_with`), encode the conceptual relationships between Content nodes and carry a confidence score. Content node can be cited by multiple Subtopics, reflecting the fact that real course concepts rarely fit a single area.

A user can edit a Content node’s display label in the frontend without breaking any citation, highlight path, or cached retrieval state, because all internal pointers use the immutable key, not the mutable label.

### 5.2 Graph Construction Pipeline

Graph construction proceeds in four stages, each implemented as a separate extractor module that emits typed `Proposal` objects rather than mutating the graph directly. The `GraphBuilder` class wraps the underlying `MultiDiGraph` and exposes `apply_proposals(bundle)` as the single mutation entry point, enforcing schema validation before any node or edge is added.

**Stage 1: Topic extraction.** The LLM is given a sample of chunk texts from the source materials and asked to identify the top-level topics covered by the course. These become the root Topic nodes of the hierarchy.

**Stage 2: Subtopic extraction.** For each Topic node, the LLM maps the full set of chunks to a list of subtopics—more granular conceptual categories that, together, cover all of the material assigned to that Topic.

**Stage 3: Content synthesis.** For each Subtopic, the LLM synthesizes a Content node from the chunks assigned to that subtopic. Each Content node carries a summary, a raw excerpt from the source, a list of key terms, and an illustration hint (an equation or diagram description that downstream components can render). Content nodes are connected to their source chunks via `EVIDENCE_OF` edges, preserving the provenance chain from a high-level concept all the way back to the original passage in the source material. Figure 2 shows the resulting Content node detail panel: the synthesized summary at the top, a rendered illustration, the list of subtopics that cite the node, and the underlying source chunks exposed for inspection.

**Stage 4: Semantic linking.** This stage adds `RELATED_TO` edges between Content nodes across different subtopics. Lumin uses a two-phase design: (a) an embedding-based pre-filter computes pairwise cosine similarities between all Content node embeddings and narrows the candidate set to those above a configurable threshold; (b) the LLM is asked to assign a `snake_case` relation label and a confidence score only to the surviving candidates. The pre-filter reduces LLM call count from quadratic to approximately linear in the number of nodes, and materially improves edge quality by preventing the model from fabricating connections between conceptually unrelated nodes.

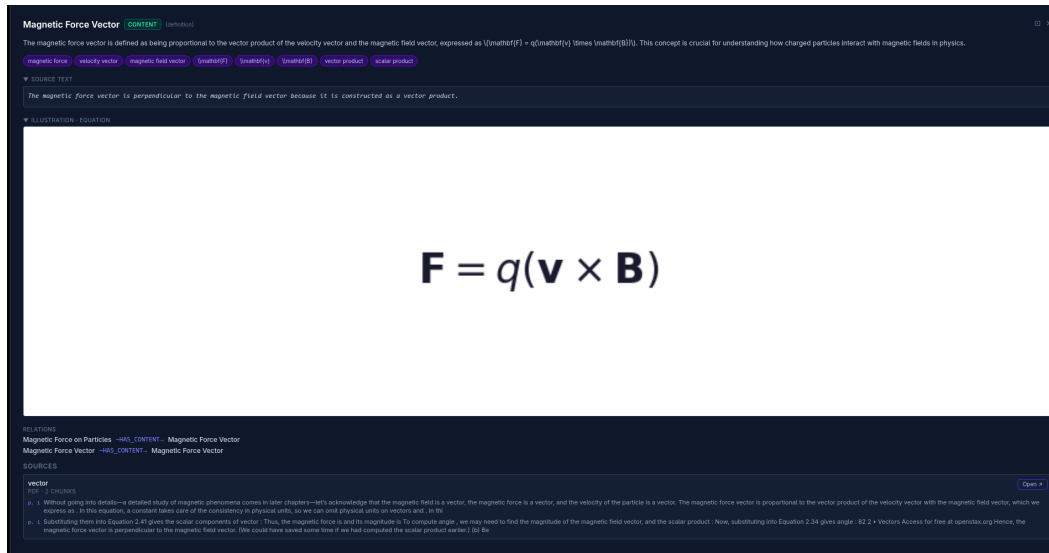


Figure 2: The Content node detail panel. Each Content node surfaces the LLM-synthesized summary, a rendered illustration (equation or diagram), the subtopics that cite it, and the raw source chunks linked via EVIDENCE\_OF edges. The provenance chain is exposed end-to-end: a student can read the synthesized concept and, in the same view, drop down into the original passage that produced it.

### 5.3 Graph Visualization

The illustration worker handles visual annotation of Content nodes. Equation hints are rendered to PNG using `matplotlib.mathtext`; diagram hints reference extracted images from the ingestion pipeline. The graph is serialized by `graph_export.py` to a `{nodes, edges}` JSON structure with `node_type` and `label` fields suitable for direct Cytoscape.js consumption on the frontend.

## 6 Retrieval and Reasoning

### 6.1 Three Retrieval Paths

Lumin’s retrieval layer resists the temptation to collapse all queries into a single code path. Three retrievers coexist and are selected by a query router:

**Dense retriever.** A FAISS IndexFlatIP index over L2-normalized sentence-transformer embeddings. Because the vectors are L2-normalized, inner product is equivalent to cosine similarity, giving maximum retrieval quality with minimum arithmetic overhead. A sidecar JSON file maps FAISS integer indices to chunk IDs. This retriever excels at surface-form lookup: finding the passage that most closely paraphrases the query in its original wording.

**Graph BFS retriever.** A relevance-guided breadth-first search over the concept graph. The retriever begins from seed nodes (Content nodes whose embeddings are most similar to the query) and expands outward along RELATED\_TO and HAS\_CONTENT edges, computing the cosine similarity between the query and each candidate neighbor’s embedding before deciding whether to expand. Expansion is pruned when the similarity of a neighbor falls below a threshold, preventing the traversal from drifting into unrelated regions of the graph. This retriever excels at relational queries: questions whose answers require synthesizing information across concepts connected by pedagogically typed edges. Figure 3 shows the subgraph produced by a typical relational query, with seed nodes and expanded neighbors visible.

**Hybrid path.** The results of the dense retriever and the graph BFS retriever are merged, de-duplicated by chunk ID, and passed together to the re-ranker. This path combines the surface-form precision of dense retrieval with the relational breadth of graph traversal, and is selected for queries that genuinely require both.

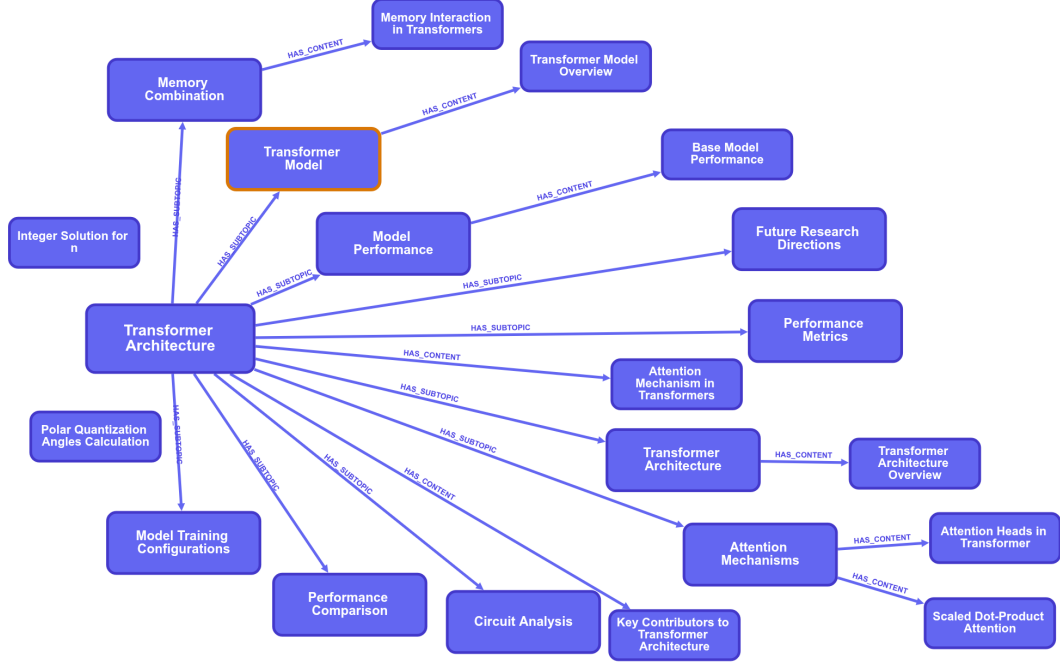


Figure 3: A retrieved subgraph for a relational query. Seed Content nodes (selected by embedding similarity to the query) anchor the traversal; relevance-guided BFS expands outward along RELATED\_TO and HAS\_CONTENT edges until similarity falls below the configured threshold. The resulting subgraph is the literal evidence trail behind the answer—each highlighted node and edge corresponds to a concrete reasoning step.

## 6.2 Query Router

The query router dispatches each incoming query to one of four outputs: `dense`, `graph`, `hybrid`, or `none`. The `none` output is an intentional feature: it lets the system answer meta-questions (“What topics are covered in this course?”) without incurring the latency and cost of retrieval.

The routing decision is made by a two-stage process. First, a cheap heuristic computes keyword overlap between the query and the names of graph nodes, and analyzes the grammatical structure of the question (questions containing relational connectives like “how does X relate to Y” favor graph retrieval; questions that closely paraphrase known terminology favor dense retrieval). When the heuristic is ambiguous, a lightweight LLM call resolves the ambiguity. This two-stage design keeps latency low for the majority of queries while remaining flexible for edge cases.

## 6.3 Re-Ranking

After retrieval, a cross-encoder re-ranker re-scores the merged candidate set. Unlike a bi-encoder, which scores query and passage independently and combines the scores, a cross-encoder attends jointly to the query and each passage, producing a much more accurate relevance judgment at the cost of higher latency. The re-ranker maps its raw logits through a sigmoid function to produce scores in  $[0, 1]$  that can be filtered directly using the per-request `min_relevancy` parameter exposed in the UI. The re-ranker is lazy-loaded so that cold-start latency is paid only when retrieval actually runs.

## 6.4 Self-RAG Reflection Gates

Every hop of Lumin’s multi-hop reasoner is wrapped in two reflection gates, both implemented as focused LLM calls that evaluate the quality of the reasoning before it proceeds.

**Pre-generation gate** (`reflect_retrieval`). Before the generator produces an answer for a given sub-question, this gate asks: *Are the retrieved chunks actually relevant to the sub-question being asked?* If the answer is no, the hop is discarded entirely, it does not enter the generation context

where it could contaminate the final answer. This is a stronger form of filtering than relevance scoring alone, because the LLM evaluates relevance in the context of the specific sub-question, not just the global query.

**Post-generation gate** (`reflect_answer`). After the generator produces an answer, this gate runs the full set of Self-RAG verdicts: ISREL (are the retrieved passages relevant?), ISSUP (is the answer supported by the passages?), ISUSE (is the answer useful?), and an aggregate OVERALL judgment. These verdicts are returned to the frontend as structured metadata and surfaced to the student via a compact `ReflectionBadge` component attached to each assistant message.

## 6.5 Multi-Hop Reasoning

The `multi_hop_reasoner` decomposes an input question into at most four sub-questions. The ceiling is a deliberate latency trade-off: beyond four serial LLM calls, response time exceeds the threshold at which students will wait. For each sub-question, the reasoner routes, retrieves, reflects, and accumulates a subgraph of visited node keys. This subgraph is returned to the frontend as `hops`, which the graph panel uses to highlight the reasoning trajectory, making the retrieval path a visible, inspectable view.

Multi-hop depth is not predicted by a learned classifier. It emerges naturally from the relevance-guided BFS, which stops expanding when similarity falls below the threshold. This design is both simpler and more robust than classifier-based alternatives, which tend to overfit to the distribution of questions seen during training.

# 7 User Interface and Visualization

## 7.1 Design Philosophy

The interface of Lumin is focused on making the AI assistant more like a tutor who is helping you navigate through your knowledge space rather than a black-box chatbot. Therefore, Lumin follows a single principle: *reasoning must be visible, inspectable, and navigable*. The interface exposes the system’s knowledge, retrieval substrate, sub-question decomposition, self-assessed confidence, and the original source of every cited claim, all within the same view.

## 7.2 Two-Pane Layout

The interface is organized as a two-pane layout (Figure ??). On the left is a Cytoscape.js-based concept graph that displays the entire knowledge base and highlights the nodes and edges that participated in answering the most recent question. On the right is a conversational column where students ask questions and read answers. The graph is not a decorative sidebar: it is an interactive map of the material that the conversation is drawing from. A student who does not trust an answer can click into any highlighted node, read its synthesized summary, and drill down to the raw evidence chunks that produced it.

## 7.3 Graph Visualization

The graph panel uses Cytoscape.js with the `fcose` force-directed layout, which handles mixed-density subgraphs gracefully. The Topic → Subtopic → Content hierarchy produces regions of very different connectivity, and `fcose`’s incremental compound-layout behaviour keeps dense clusters legible without shrinking sparse branches into invisibility.

Node classes—topic, subtopic, content, chunk\_ref, drive size and color through a Cytoscape stylesheet rather than per-node styling, keeping visual logic out of component code. Topics dominate the canvas; subtopics orbit them; content nodes form dense fringes; evidence chunks appear only when a user explicitly expands a branch. Collapse/expand traverses the HAS\_SUBTOPIC and HAS\_CONTENT edges downward from Topic roots, giving students local control over visual density. Without this affordance, a graph with several hundred content nodes becomes a “hairball” that conveys nothing; with it, students can incrementally reveal detail in the region they care about.

## 7.4 Evidence, Citations, and Source Deep-Linking

Every answer rendered in the chat pane carries two companions: colored source badges indicating the retrieval path that produced it (dense, graph, or hybrid), and a row of clickable evidence chips. The source badges are more than decoration: they give students intuition about which retrieval mode handles which kinds of questions, and inform when a power user might want to override the router's default choice.

Clicking an evidence chip opens the raw chunk text—the specific passage that grounded the corresponding claim. For modalities with natural addressing, the chip also exposes a deep-link into the source file. The `utils/source.ts` helper constructs URLs of the form `/api/source/<file>?page=3` for PDFs and `?t=42s` for video/audio, leveraging the browser's native PDF and media players to jump directly to the cited moment. This is a deliberately low-technology decision: rather than building a custom in-page viewer, Lumin relies on well-established browser primitives that students already know how to use. Figure 2 shows the same provenance chain exposed in the Content node detail panel, where evidence chunks are listed beneath the synthesized summary.

## 7.5 Self-Reflection Badges

The Self-RAG reflection verdicts (ISREL, ISSUP, ISUSE, and OVERALL) are attached to each assistant message, exposing the system's internal reasoning to the student's discretion. This feature allows a student to see at a glance when the system regards its own answer as under-supported, and treat the answer accordingly rather than internalizing a flawed explanation. In educational contexts, exposed uncertainty is far preferable to confident wrongness.

## 7.6 Quiz Generation

Beyond question answering, Lumin includes a quiz generation panel (Figure 4) that turns the same knowledge space into an active-recall study tool. A student can type its focus area or allow Lumin to use entire knowledge space to produce multiple-choice, short-answer or True/False questions grounded in the corresponding evidence chunks. Because each question is generated from a specific chunk, every item is traceable: the panel exposes the same evidence chips used in the chat panel, so a student who answers incorrectly can immediately inspect the source passage that contains the correct information. This allows students to use Lumin not only for answering questions and supporting research, but also as a tool for studying.

## 7.7 Query Controls

The query input bar exposes controls that most consumer chat interfaces hide. A routing mode selector (auto / dense / graph / hybrid) lets advanced users override the router. A gear popover reveals per-request `min_relevancy` and `max_sources` parameters, which control the re-ranker's cutoff behavior. These controls are gated behind a popover so casual users are not overwhelmed, but are present for power users and for evaluators who want to compare retrieval modes on identical queries.

## 7.8 Math and Code Rendering

Course material is saturated with equations and code. The `RichText` component composes `react-markdown`, `remark-math`, and `rehype-katex` into a single renderer used across the chat pane and the node detail panel. Using one renderer throughout guarantees that an equation looks identical whether it appears in a generated answer, a Content node summary, or an evidence chip—a consistency that matters when a student is visually tracking a formula across the interface to verify that the system has not silently paraphrased it.

## 7.9 Request Lifecycle

A query follows a fixed sequence through the stack. The frontend posts to `/api/query`; the Self-RAG reasoner decomposes the question into sub-questions and emits a `RETRIEVE` token if the query is knowledge-intensive. On `RETRIEVE`, the dense retriever and graph BFS retriever run in parallel; their

Figure 4: The quiz generation panel. Questions are produced from knowledge space, so a student who answers incorrectly can drop directly into the source passage. Lumin thus serves dual purposes: for retrieval and for self-assessment.

candidate sets are merged, de-duplicated by chunk ID, and passed to the cross-encoder re-ranker. The top- $k$  reranked chunks, together with the subgraph expanded around their source nodes, are handed to the generator. A second reflection pass scores each generated sentence for support; sentences below the support threshold trigger either re-retrieval with a refined query or a hedged phrasing. The final payload contains the answer text, cited chunk IDs, and the serialized subgraph for the mind-map view.

## 8 Limitations and Future Work

Lumin in its current form has clear limits that define the direction of future work.

**Single-tenant knowledge base.** The current graph is monolithic: one `MultiDiGraph`, one FAISS index, one SQLite store. Students realistically manage multiple simultaneous courses and projects, each of which deserves its own isolated knowledge graph. Future work will introduce a partition key throughout the persistence layer, with per-space embeddings and a lightweight management UI for creating, renaming, and switching between spaces.

**Passive ingestion.** Lumin’s knowledge base is authored implicitly, one uploaded file at a time. A more powerful mode would allow a student to specify a high-level topic “transformer architectures,” “stochastic processes” or a research question and have a team of specialist agents (a papers agent, a textbooks agent, a survey agent) assemble and ingest a complete knowledge base on their behalf, seeding the graph with a curriculum-shaped outline before the LLM extractors run.

**Graph maintenance.** Over time, any graph built by LLM extraction accumulates artifacts: near-duplicate Content nodes, overly granular Subtopics, and edges whose labels no longer fit the accumu-

lated material. A background maintenance agent could periodically propose consolidations and apply them after a reflection pass or explicit user approval, using the deterministic node keys to rewrite edges transactionally while preserving evidence provenance.

## 9 Conclusion

This paper introduced Lumin, a proof of concept of an academic assistant. Lumin’s three contributions work together toward this goal. The knowledge graph organizes course materials in terms of concepts and their relations. The adaptive retrieval router, gated by SELF-RAG reflection, grounds the AI in the student’s sources. And the interactive mind map renders the reasoning path as a navigable artifact, giving students the means to verify, investigate, and explore every step of the system’s thinking.

A key advantage of this system is the ability for students to organize their multimodal sources in one place, ensuring they can better take advantage of rich resources to aid their studies. Our work provides a potential system that can be implemented in the future as a better alternative to standard chatbots.

However, several limitations of this system must be discussed. This study currently does not have any way to evaluate its effectiveness other than informal use and observations by a limited number of students. Furthermore, the system has not eliminated hallucinations due to the stochastic nature of the underlying generator. When the self-reflection tokens catch and report them, this still does not help the students’ purpose for using this tool. This shows the necessity for a totally different architectural model that eliminates hallucinations. Regardless, the system introduced here offers a strong, systematic way to create grounded and helpful AI learning assistants.

## Acknowledgments

I would like to express my gratitude to Professor Fares Fraij for his mentorship and feedback throughout this project. His guidance shaped both the direction of the research and the concrete implementation decisions described in this paper, and his patience and encouragement at every stage made this project possible.

## References

- [1] Digital Education Council, “What students want: Key results from DEC Global AI Student Survey 2024,” <https://www.digitaleducationcouncil.com/post/what-students-want-key-results-from-dec-global-ai-student-survey-2024>. 2024.
- [2] College Board, “New research: Majority of high school students use generative AI for schoolwork,” <https://newsroom.collegeboard.org/new-research-majority-high-school-students-use-generative-ai-schoolwork>, also available at [https://research.collegeboard.org/media/pdf/ai-research-brief-1\\_vf.pdf](https://research.collegeboard.org/media/pdf/ai-research-brief-1_vf.pdf). 2025.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [4] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

- [7] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document ranking with a pretrained sequence-to-sequence model. In *Findings of EMNLP*, 2020.
- [8] Joseph D. Novak and Alberto J. Cañas. The theory underlying concept maps and how to construct and use them. Technical report, Institute for Human and Machine Cognition, 2010.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [10] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of ICML*, 2023.